COM115 Week 6

Matrices

Refer to the Python code examples in the lecture notes, and write code to answer the following questions.

Enter your code for each Exercise in the cell beneath it. You can add additional code cells if you prefer.

In [1]:
```
#run this cell to import numpy
import numpy as np
```

Exercise 1

Let

$$A = \begin{bmatrix} 1 & 4 \\ 3 & -1 \\ 0 & 7 \end{bmatrix}, B = \begin{bmatrix} 2 & 1 & 3 \\ -5 & 6 & 1 \end{bmatrix}, C = \begin{bmatrix} -3 \\ 9 \\ 0 \\ 9 \end{bmatrix}, D = \begin{bmatrix} 2 & 1 \\ -3 & 0 \end{bmatrix}$$

(a) Represent each matrix in Python and determine its order.

(b) Display the transpose of each matrix.

(c) What is $A + B^T$?

(d) What is $D^T - 2D$?

In [2]:
```
import numpy as np

# (a)
A = np.array([[1,4],[3,-1],[0,7]])
print('A\n',A)
print('Order of A',A.shape)

B = np.array([[2,1,3],[-5,6,1]])
print('\nB\n',B)
print('Order of B',B.shape)

C = np.array([[-3],[9],[0],[9]])
print('\nC\n',C)
print('Order of C',C.shape)

D = np.array([[2,1],[-3,0]])
print('\nD\n',D)
print('Order of D',D.shape)

# (b)
print('\nA^T\n',A.T)
print('\nB^T\n',B.T)
print('\nC^T\n',C.T)
```

```python
print('\nD^T\n',D.T)

# (c)
print('\nA + B^T')
print(A + B.T)

# (d)
print('\nD^T -2D')
print(D.T -2*D)
```

```
A
 [[ 1  4]
 [ 3 -1]
 [ 0  7]]
Order of A (3, 2)

B
 [[ 2  1  3]
 [-5  6  1]]
Order of B (2, 3)

C
 [[-3]
 [ 9]
 [ 0]
 [ 9]]
Order of C (4, 1)

D
 [[ 2  1]
 [-3  0]]
Order of D (2, 2)

A^T
 [[ 1  3  0]
 [ 4 -1  7]]

B^T
 [[ 2 -5]
 [ 1  6]
 [ 3  1]]

C^T
 [[-3  9  0  9]]

D^T
 [[ 2 -3]
 [ 1  0]]

A + B^T
[[ 3 -1]
 [ 4  5]
 [ 3  8]]

D^T -2D
[[-2 -5]
 [ 7  0]]
```

Exercise 2

If possible carry out the following matrix multiplications based on the matrices from Exercise 1.

(a) BD

(b) $D^T A^T$

(c) CC^T

(d) D^2, D^3, D^4, D^5

(e) Show that $AB \neq BA$

(f) Show that $(AB)^T = B^T A^T$

Let

$$E = \begin{bmatrix} 2 & 2 \\ 1 & 5 \\ 4 & 3 \end{bmatrix}$$

(g) Show that $A(BE) = (AB)E$

In [3]:
```
# (a)
print('Order of B',B.shape)
print('Order of D',D.shape)
print('Order of B and D is incompatible for multiplication.')

# (b)
print('\nD^TA^T\n',D.T @ A.T)

# (c)
print('\nCC^T\n',C @ C.T)

# (d)
print('\nD^2\n',np.linalg.matrix_power(D,2))
print('\nD^3\n',np.linalg.matrix_power(D,3))
print('\nD^4\n',np.linalg.matrix_power(D,4))
print('\nD^5\n',np.linalg.matrix_power(D,5))

# (e)
print('\nAB\n',A @ B)
print('\nBA\n',B @ A)
print('Clearly AB and BA are different.')

# (f)
print('\n(AB)^T\n',(A @ B).T)
print('\nB^TA^T\n',B.T @ A.T)
print('Clearly (AB)^T = B^TA^T.')

# (g)
E = np.array([[2,2],[1,5],[4,3]])
print('\nA(BE)\n',A @ (B @ E))
print('\n(AB)E\n',(A @ B) @ E)
print('Clearly A(BE) = (AB)E.')
```

```
Order of B (2, 3)
Order of D (2, 2)
Order of B and D is incompatible for multiplication.

D^TA^T
 [[-10   9 -21]
 [  1   3   0]]

CC^T
 [[  9 -27   0 -27]
 [-27  81   0  81]
 [  0   0   0   0]
 [-27  81   0  81]]

D^2
 [[ 1  2]
 [-6 -3]]

D^3
 [[-4  1]
 [-3 -6]]

D^4
 [[-11  -4]
 [ 12  -3]]

D^5
 [[-10 -11]
 [ 33  12]]

AB
 [[-18  25   7]
 [ 11  -3   8]
 [-35  42   7]]

BA
 [[  5  28]
 [ 13 -19]]
Clearly AB and BA are different.

(AB)^T
 [[-18  11 -35]
 [ 25  -3  42]
 [  7   8   7]]

B^TA^T
 [[-18  11 -35]
 [ 25  -3  42]
 [  7   8   7]]
Clearly (AB)^T = B^TA^T.

A(BE)
 [[ 17 110]
 [ 51  31]
 [  0 161]]

(AB)E
 [[ 17 110]
 [ 51  31]
 [  0 161]]
Clearly A(BE) = (AB)E.
```

Exercise 3

Show how you would create the following matrices using special functions rather than having to list all the values in your code.

(a) $G = \begin{bmatrix} 7 & 7 & 7 & 7 & 7 \\ 7 & 7 & 7 & 7 & 7 \\ 7 & 7 & 7 & 7 & 7 \\ 7 & 7 & 7 & 7 & 7 \\ 7 & 7 & 7 & 7 & 7 \end{bmatrix}$

In [4]:
```
G = 7 * np.ones((5,5))
print('G\n',G)
```

```
G
 [[7. 7. 7. 7. 7.]
 [7. 7. 7. 7. 7.]
 [7. 7. 7. 7. 7.]
 [7. 7. 7. 7. 7.]
 [7. 7. 7. 7. 7.]]
```

(b) $H = \begin{bmatrix} 2 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \\ -1 & 2 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & 2 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & 2 & -1 & -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & 2 & -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 & 2 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 & -1 & 2 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 & -1 & -1 & 2 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & 2 & -1 \\ -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & 2 \end{bmatrix}$

In [5]:
```
H1 = -1 * np.ones((10,10))
H2 = 3 * np.eye(10)
H = H1 + H2
print('\nH\n',H)
```

```
H
 [[ 2. -1. -1. -1. -1. -1. -1. -1. -1. -1.]
 [-1.  2. -1. -1. -1. -1. -1. -1. -1. -1.]
 [-1. -1.  2. -1. -1. -1. -1. -1. -1. -1.]
 [-1. -1. -1.  2. -1. -1. -1. -1. -1. -1.]
 [-1. -1. -1. -1.  2. -1. -1. -1. -1. -1.]
 [-1. -1. -1. -1. -1.  2. -1. -1. -1. -1.]
 [-1. -1. -1. -1. -1. -1.  2. -1. -1. -1.]
 [-1. -1. -1. -1. -1. -1. -1.  2. -1. -1.]
 [-1. -1. -1. -1. -1. -1. -1. -1.  2. -1.]
 [-1. -1. -1. -1. -1. -1. -1. -1. -1.  2.]]
```

(c) $K = \begin{bmatrix} 2 & 1 & 0 & 0 & 0 & 0 \\ 1 & 2 & 1 & 0 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 \\ 0 & 0 & 1 & 2 & 1 & 0 \\ 0 & 0 & 0 & 1 & 2 & 1 \\ 0 & 0 & 0 & 0 & 1 & 2 \end{bmatrix}$

In [6]:
```
K = 2 * np.eye(6)
K[1:6,0:5] = K[1:6,0:5] + np.eye(5)
K[0:5,1:6] = K[0:5,1:6] + np.eye(5)
print('\nK\n',K)
```

```
K
 [[2. 1. 0. 0. 0. 0.]
 [1. 2. 1. 0. 0. 0.]
 [0. 1. 2. 1. 0. 0.]
 [0. 0. 1. 2. 1. 0.]
 [0. 0. 0. 1. 2. 1.]
 [0. 0. 0. 0. 1. 2.]]
```

Exercise 4

(a) Find the determinant of each of the matrices in Exercise 3.

In [9]:
```
# (a)
print('Determinant of G')
print(np.linalg.det(G))

print('\nDeterminant of H')
print(np.linalg.det(H))
```

```
Determinant of G
0.0

Determinant of H
-137781.00000000003
```

(b) If possible, find the inverse of the matrices G and H in Exercise 3.

In [11]:
```
# (b)
print('\nCannot compute the inverse of G as it is singular.')

H_inv = np.linalg.inv(H)
print('\nInverse of H')
print(H_inv)

# To check we can compute H * H_inv
print('\nCheck H x H_inv = I')
print(H @ H_inv)
```

```
Cannot compute the inverse of G as it is singular.

Inverse of H
[[ 0.28571429 -0.04761905 -0.04761905 -0.04761905 -0.04761905 -0.04761905
  -0.04761905 -0.04761905 -0.04761905 -0.04761905]
 [-0.04761905  0.28571429 -0.04761905 -0.04761905 -0.04761905 -0.04761905
  -0.04761905 -0.04761905 -0.04761905 -0.04761905]
 [-0.04761905 -0.04761905  0.28571429 -0.04761905 -0.04761905 -0.04761905
  -0.04761905 -0.04761905 -0.04761905 -0.04761905]
 [-0.04761905 -0.04761905 -0.04761905  0.28571429 -0.04761905 -0.04761905
  -0.04761905 -0.04761905 -0.04761905 -0.04761905]
 [-0.04761905 -0.04761905 -0.04761905 -0.04761905  0.28571429 -0.04761905
  -0.04761905 -0.04761905 -0.04761905 -0.04761905]
 [-0.04761905 -0.04761905 -0.04761905 -0.04761905 -0.04761905  0.28571429
  -0.04761905 -0.04761905 -0.04761905 -0.04761905]
 [-0.04761905 -0.04761905 -0.04761905 -0.04761905 -0.04761905 -0.04761905
   0.28571429 -0.04761905 -0.04761905 -0.04761905]
 [-0.04761905 -0.04761905 -0.04761905 -0.04761905 -0.04761905 -0.04761905
  -0.04761905  0.28571429 -0.04761905 -0.04761905]
 [-0.04761905 -0.04761905 -0.04761905 -0.04761905 -0.04761905 -0.04761905
  -0.04761905 -0.04761905  0.28571429 -0.04761905]
 [-0.04761905 -0.04761905 -0.04761905 -0.04761905 -0.04761905 -0.04761905
  -0.04761905 -0.04761905 -0.04761905  0.28571429]]

Check H x H_inv = I
[[ 1.00000000e+00 -5.55111512e-17  0.00000000e+00  0.00000000e+00
   0.00000000e+00  1.38777878e-17  1.38777878e-17  1.38777878e-17
   0.00000000e+00  0.00000000e+00]
 [-5.55111512e-17  1.00000000e+00 -5.55111512e-17 -5.55111512e-17
   5.55111512e-17  1.38777878e-17 -4.16333634e-17 -4.16333634e-17
   0.00000000e+00  0.00000000e+00]
 [ 1.11022302e-16 -5.55111512e-17  1.00000000e+00  5.55111512e-17
  -5.55111512e-17  1.38777878e-17  1.38777878e-17  1.38777878e-17
   0.00000000e+00  0.00000000e+00]
 [ 1.11022302e-16 -5.55111512e-17  5.55111512e-17  1.00000000e+00
  -1.11022302e-16  1.38777878e-17  1.38777878e-17  1.38777878e-17
   0.00000000e+00  0.00000000e+00]
 [ 1.11022302e-16 -5.55111512e-17  5.55111512e-17  5.55111512e-17
   1.00000000e+00  1.38777878e-17 -1.38777878e-17  1.38777878e-17
   0.00000000e+00  0.00000000e+00]
 [ 1.11022302e-16 -5.55111512e-17  5.55111512e-17  5.55111512e-17
  -3.33066907e-16  1.00000000e+00  1.38777878e-17 -4.16333634e-17
   0.00000000e+00  0.00000000e+00]
 [ 8.32667268e-17 -8.32667268e-17  5.55111512e-17  5.55111512e-17
  -3.60822483e-16 -6.93889390e-17  1.00000000e+00 -1.38777878e-17
   0.00000000e+00  0.00000000e+00]
 [ 9.71445147e-17 -6.93889390e-17  6.93889390e-17  6.93889390e-17
  -3.46944695e-16 -5.55111512e-17  2.77555756e-17  1.00000000e+00
   1.38777878e-17  0.00000000e+00]
 [ 1.31838984e-16 -3.46944695e-17  1.04083409e-16  1.04083409e-16
  -3.12250226e-16 -9.71445147e-17 -1.38777878e-17 -6.93889390e-18
   1.00000000e+00  1.38777878e-17]
 [ 1.66533454e-16  0.00000000e+00  1.38777878e-16  1.38777878e-16
  -2.77555756e-16 -9.71445147e-17 -1.38777878e-17 -1.38777878e-17
   0.00000000e+00  1.00000000e+00]]
```

Exercise 5

Solve the following set of linear equations for x, y and z:

$$x + y + z = 3 \tag{1}$$
$$2x + 7y + 2z = 15 \tag{2}$$
$$-3x + 4y + z = 0 \tag{3}$$

In [8]:
```
A2 = np.array([[1,1,1],[2,7,2],[-3,4,1]])
b2 = np.array([[3,15,0]]).T
print('A\n',A2)
print('b\n',b2)

v2 = np.linalg.solve(A2,b2)
print('v')
print(v2)
```

```
A
 [[ 1  1  1]
 [ 2  7  2]
 [-3  4  1]]
b
 [[ 3]
 [15]
 [ 0]]
v
[[ 2.1]
 [ 1.8]
 [-0.9]]
```